CDC Replay Correctness Lab: Making Stable but Incorrect State Visible

Abstract

Change-data capture consumers can produce a stable but incorrect materialization: identity deduplication does not order distinct changes, and finite identity memory may hide conflicting historical payloads without changing the latest state. CDC Replay Correctness Lab is a local, interactive demonstration of these distinctions. Reviewers select ordering, replay policy, checkpoint and interruption boundaries and ID retention, inspect event-by-event decisions, and compare actual state with an independent canonical oracle. A guaranteed stale-delete preset reproduces baseline failure without searching seeds. Independent execution of the original artifact reproduced its 126 principal and 36 bounded observations. New isolated controls show that all three policies pass duplicate-only and ordered delete/reinsert workloads; an ID-only policy still fails the stale-delete case. A separate probe-order study exposes why retention-dependent conflict detection cannot be inferred from final-state equality. The contribution is an inspectable, reproducible educational and diagnostic artifact, not a new CDC algorithm or an exactly-once guarantee.

CCS Concepts

• **Software and its engineering** → **Software testing and debugging**; • **Information systems** → *Data replication tools.*

Keywords

change-data capture, replay, correctness oracle, interactive demonstration

1 Problem and demonstration contribution

A downstream materializer can receive a historical deletion after a newer reinsertion. Keeping only unique event IDs does not prevent this stale overwrite when the IDs are distinct. Moreover, a second replay may leave an already incorrect state unchanged. A demonstration that reports only duplicate counts or redelivery stability can therefore conceal the failure it is supposed to explain.

The lab turns these counterexamples into an audience-controlled comparison of three simple policies. It separates *final-state equality*, *duplicate recognition*, *conflict detection*, and *recovery attempts*. A reviewer can switch from whole-input validation to online processing and expire identities while retaining versioned tombstones. This reveals both containment and missing detection rather than displaying only favorable outcomes. The artifact makes established semantics observable; it introduces no new fault-tolerance algorithm and reports no user-study or production result.

The event contract assumes immutable identity, an authoritative comparable per-key integer version, a string payload and a deletion flag. Higher versions determine the intended state; a latest deleted record remains a tombstone until superseded. Real sources require source-specific ordering contracts: Debezium's PostgreSQL documentation describes LSN-bearing change events and offset-based restart [5]. We neither derive comparable versions from arbitrary CDC records nor implement that connector.

SIGMOD 2027 Demonstrations,
2027.

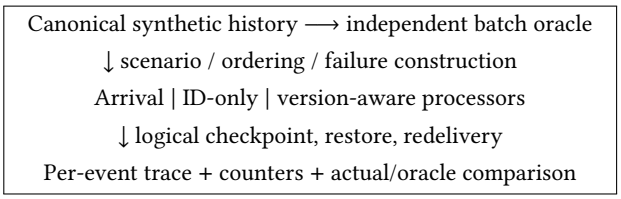


Figure 1: The canonical specification and replay transitions are separate. The interface shares the tested model with the V2 experiment runner.

2 Architecture and correctness model

The system has a Python HTTP server from the standard library, embedded HTML and JavaScript, a replay model, two research runners and artifact validators. It needs no external service or front-end download. The installed wheel serves the same interface from outside the checkout. Requests describe a bounded synthetic scenario; they do not upload arbitrary client data. State is local to one request.

The independently implemented oracle groups the canonical history by key and selects the maximum version. Its identity and key/version ambiguity checks are separately implemented; it calls no materializer transitions or shared processor conflict validator. It is computed before injected delivery is processed. Hand-authored tests assert stale updates, deletion/reinsertion and ambiguous identities/versions. This is a test specification, not an external database authority or formal proof.

Arrival-order replay overwrites on each event. ID-only replay suppresses remembered IDs but otherwise overwrites. Version-aware replay also refuses lower-version state writes and conflicting equal-version values. Principal ID-based policies use an unlimited ID ledger; arrival replay keeps none. A shared preflight validator rejects conflicting input before any of the three policies executes; that rejection must not be called online conflict detection.

At checkpoint offset c, logical state and ID memory are JSON-serialized. Execution continues to interruption i, discards later attempts, restores the earlier snapshot and replays from c. These are simulated logical checkpoints: no disk write, fsync, transactional source/sink coupling or real crash is provided. Distributed consistent cuts are a substantially stronger problem [4].

For completed recovery runs, the trace marks events attempted before interruption that are discarded when the snapshot is restored. Monotonicity counters separate attempted writes, committed-prefix writes and recovered writes. Restore can lower a visible version because an earlier snapshot is reinstated; it is not counted as a stale materialization write. A completed run's correctness requires whole-state oracle equality, independently of its redelivery stability.

Stale-delete preset: one key, native order

`b0:v4`: live v4 ⟶ `b0:v3`: delete v3

Policy	Actual	Oracle
Arrival	deleted v3	live v4
ID-only	deleted v3	live v4
Version-aware	live v4	live v4

Inspect: strategy switch; wrong-key reason; trace position; before/after state; stale-write decision.

Figure 2: Readable typeset view of verified preset outputs, not a browser screenshot. The working interface exposes these states and interactions; the artifact retains the full Chromium screenshot. Distinct IDs do not protect against stale deletion.

3 Live demonstration experience

The interface exposes nine scenarios, policy selection, native, reversed, seeded or version-based ordering, logical checkpoint and interruption offsets, and finite or unlimited ID capacity. Side-by-side states and wrong-key explanations identify a missing key, stale version, unexpected version, or payload/tombstone mismatch. Trace rows show phase, source offset, event identity, applied, duplicate, stale or conflict decisions and ledger size. A slider reveals state after the selected transition. Invalid controls produce an explicit error; rejected or conflict-stopped runs have no completed correctness verdict.

The five-minute walkthrough follows five interactions:

(1) Click the stale-delete preset: one key receives live v4 under one ID and deleted v3 under another. Arrival-order replay visibly fails within the automated one-minute check. This case is deterministic and requires no seed search.
(2) Select ID-only, then version-aware. ID-only remains wrong; version-aware keeps live v4. Inspect the stale decision and before/after state with the trace controls.
(3) Choose recovery/redelivery, three keys, checkpoint 4 and interruption 8. Inspect four discarded attempts and four replayed lost attempts; compare final state and the distinct redelivery-stability indicator.
(4) Choose ledger expiry, 16 keys, online validation and capacity 4. Latest state matches the oracle, yet an injected conflicting stale ID payload is not detected; the interface explicitly warns about this distinction.
(5) Clear the capacity for unlimited retention. The same conflict is detected and processing stops. Switch to whole-input validation to see rejection before replay.

The allocation is a presenter script, not an observed five-minute usability study. HTTP integration tests cover preflight rejection. Real Chromium automation executes the preset, policy switches, trace stepping, recovery and finite and unlimited ledger interactions with no JavaScript errors. The browser uses the built wheel outside the repository.

Table 1: V2 outcomes per policy: 36 completed and 12 preflight-rejected inputs.

Policy	Correct	Incorrect	Rejected
Arrival	24	12	12
ID-only	30	6	12
Version-aware	36	0	12

4 Evidence and isolated controls

The protocol was repository-committed before implementation and measurements. A new execution of the unchanged original runner reproduced all non-timing fields of its 126 principal and 36 bounded observations. Artifact validation checked original and new SHA-256 manifests, CSV/JSON field parity, source and configuration hashes and row counts. This is independent execution, not replication by a separately authored implementation. Timing samples were intentionally not required to match.

Original duplicate and delete/reinsert conditions also shuffled input; they were compound workloads. V2 adds truly isolated controls and records a new 144-row matrix: 8/64 keys, three seeds, eight conditions, three policies, five fresh timed executions. Some ordered conditions do not vary with seed; rows are not independent statistical samples. Conflicting-ID and same-version conditions yield 12 rejected inputs per policy. The remaining 36 completed observations per policy are summarized below.

All policies pass all six configurations each of ordered live input, duplicate-only, ordered delete/reinsert and ordered checkpoint recovery. Thus neither duplicates alone nor an ordered deletion/reinsertion sequence establishes an advantage for version guards in this final-state model. Arrival and ID-only fail all six stale-delete configurations. Reverse post-recovery redelivery additionally makes arrival replay wrong; remembered identities contain that case for ID-only.

A separate 72-row online matrix crosses policy, capacities of 0, 4, 16 or unlimited, three recorded seeds and two probe orders. State and tombstones remain unbounded. In *conflict-first*, version-aware replay detects the expired-ID conflict only with unlimited memory. In *exact-then-conflict*, capacities 4/16 detect it after exact stale redelivery reinserts the ID; capacity 0 still misses it. Each configuration is repeated over three seeds, but its ordered bootstrap is identical. A version guard can preserve latest state while losing historical-payload consistency checks.

Five timing samples and per-row medians are retained for every observation. Principal run timing includes validation, snapshot copies, serialization and event traces; expiry timing covers the materializer bootstrap and probes, without traces or checkpointing. Both exclude data construction, oracle/scoring and output I/O. Trace copying changes measured work, so V2 timings are not used as speedups over the original runner. One host, small synthetic streams and early rejection preclude throughput, cloud-cost or production conclusions.

5 Related work, limits and reproducibility

MillWheel studies persistent state and fault-tolerant streaming [1]; the Dataflow Model treats correctness, latency and cost for unordered input [2]. This lab does not reproduce either system. Its reference-specification approach sits within established model-based testing [7]; it does not benchmark against that taxonomy or introduce a new test-oracle method. Interactive algorithm visualizations such as RaftScope [6] predate this work. Our specific demonstration joins CDC stale-delete counterexamples, attempted and recovered trace evidence and probe-order-dependent ID expiry, rather than claiming invention of visualization, idempotency, versions or checkpoints.

The model has no distributed scheduler, external CDC connector, durable sink, concurrency, source epochs or tombstone garbage collection. Stable IDs and authoritative versions are assumptions. Equal versions cannot resolve ambiguous data. Final-state equality does not measure external side-effect multiplicity or imply exactly-once processing. A finite ID ledger is not a bound on total state. Claims are limited to retained deterministic workloads; independent implementation replication and a broader prior-art review remain outstanding.

The public artifact [3] provides unit tests, frozen V4/V5 validation, fresh-output commands, retained evidence and the five-minute walkthrough. Start the interface using Python's `-m` option with module `pipeline_reliability.cdc_interactive`. Artifacts retain the source commit, dirty status, software environment, seeds, configuration, timing samples and hashes.

References

[1] Tyler Akidau, Alex Balikov, Kaya Bekiroglu, Slava Chernyak, Josh Haberman, Reuven Lax, Sam McVeety, Daniel Mills, Paul Nordstrom, and Sam Whittle. 2013. MillWheel: Fault-Tolerant Stream Processing at Internet Scale. *Proceedings of the VLDB Endowment* 6, 11 (2013), 1033–1044. https://www.vldb.org/pvldb/vol6/p1033-akidau.pdf

[2] Tyler Akidau, Robert Bradshaw, Craig Chambers, Slava Chernyak, Rafael J. Fernandez-Moctezuma, Reuven Lax, Sam McVeety, Daniel Mills, Frances Perry, Eric Schmidt, and Sam Whittle. 2015. The Dataflow Model: A Practical Approach to Balancing Correctness, Latency, and Cost in Massive-Scale, Unbounded, Out-of-Order Data Processing. *Proceedings of the VLDB Endowment* 8, 12 (2015), 1792–1803. doi:10.14778/2824032.2824076

[3] CDC Replay Correctness Lab repository contributors. [n. d.]. CDC Replay Correctness Lab: Reproducible Artifact. https://github.com/priyankaVelu/gcp-data-pipeline-reliability-kit/tree/583b90bb5a8bc33b99e4812c4a29399d416beb19/research/cdc_lab/v2 Immutable main snapshot; README, protocol, results, and walkthrough. Accessed 2026-10-29.

[4] K. Mani Chandy and Leslie Lamport. 1985. Distributed Snapshots: Determining Global States of Distributed Systems. *ACM Transactions on Computer Systems* 3, 1 (1985), 63–75. doi:10.1145/214451.214456

[5] Debezium contributors. [n. d.]. Debezium Connector for PostgreSQL. Primary connector documentation, revision d9e8eeb71f617586ac2cff41ebef2ca7f4ee308e. https://github.com/debezium/debezium/blob/d9e8eeb71f617586ac2cff41ebef2ca7f4ee308e/documentation/modules/ROOT/pages/connectors/postgresql.adoc Accessed 2026-10-13.

[6] RaftScope contributors. [n. d.]. RaftScope: Raft Visualization. Primary project repository, revision 5b0c10ab51f873721895e7470b49e04c94bf826f. https://github.com/ongardie/raftscope/tree/5b0c10ab51f873721895e7470b49e04c94bf826f Accessed 2026-10-13.

[7] Mark Utting, Alexander Pretschner, and Bruno Legeard. 2012. A Taxonomy of Model-Based Testing Approaches. *Software Testing, Verification and Reliability* 22, 5 (2012), 297–312. doi:10.1002/stvr.456